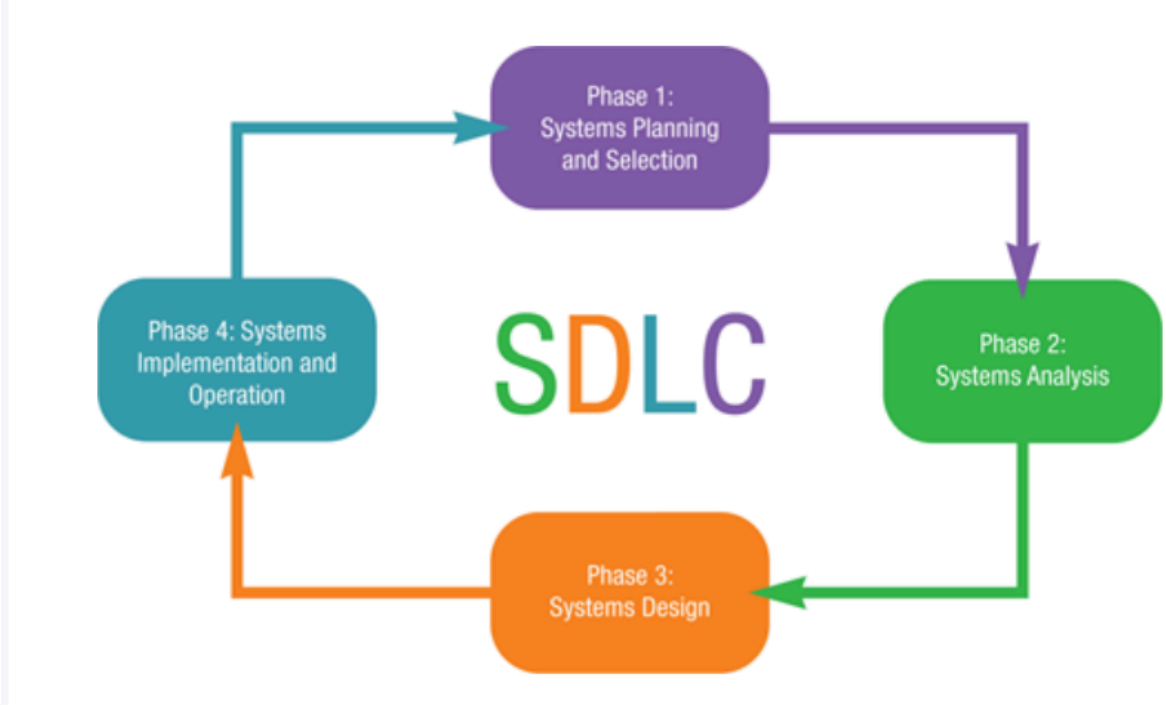
MIS 3373
Systems Analysis and Design Theory
Spring 2025
Toolbox

Chisanshi Chiwele

WEEK 1

4 Steps of the SDLC



1. ***Systems Planning and Selection***- an organization's total information system needs are analyzed and arranged, and a potential information systems project is identified and an argument for continuing or not continuing with the project is presented
2. ***Systems Analysis***- the current system is studied and alternative replacement systems are proposed.
3. ***Systems Design***- the system chosen for development in systems analysis is first described independently of any computer platform (logical design), and is then transformed into technology-specific details (physical design) from which all programming and system construction can be accomplished.
4. ***Systems Implementation and Operation***- the information system is coded, tested, and installed in the organization, and the information system is systematically repaired and improved

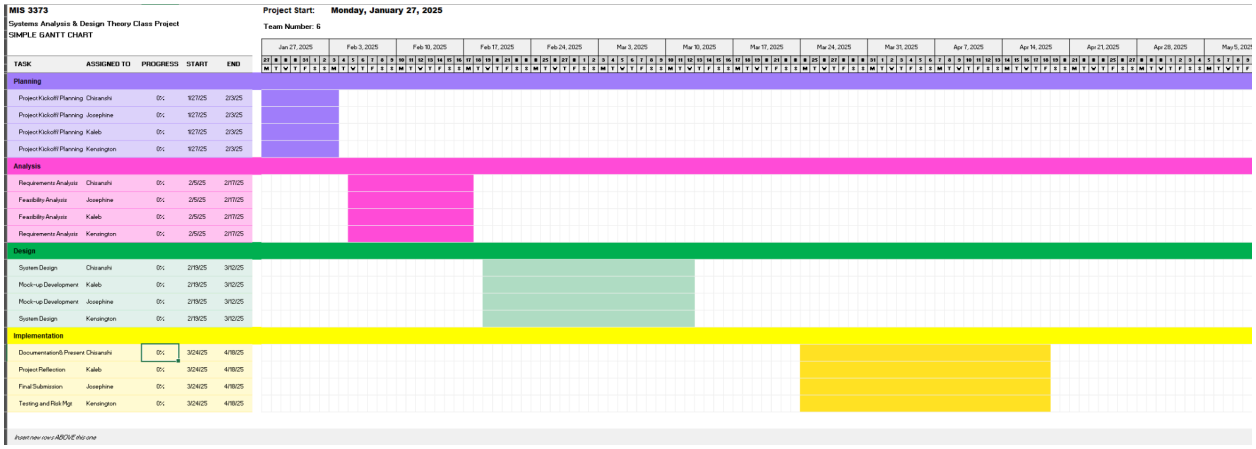
Key Terms and Definitions from Chapter 1

1. ***Agile Methodologies***- A family of development methodologies characterized by short iterative cycles and extensive testing; active involvement of users for establishing, prioritizing, and verifying requirements; and a focus on small teams of talented, experienced programmers.
2. ***Application software***- Software designed to process data and support users in an organization. Examples include spreadsheets, word processors, and database management systems.
3. ***Boundary***- The line that marks the inside and outside of a system and that sets off the system from its environment.
4. ***Cohesion***- The extent to which a system or subsystem performs a single function.
5. ***Component***- An irreducible part or aggregation of parts that makes up a system; also called a subsystem.
6. ***Computer-aided software engineering (CASE)***- Software tools that provide automated support for some portion of the systems development process.
7. ***Constraint***- A limit to what a system can accomplish.
8. ***Coupling***- The extent to which subsystems depend on each other.
9. ***Decomposition***- The process of breaking the description of a system down into small components; also known as functional decomposition.
10. ***Environment***- Everything external to a system that interacts with the system.
11. ***Information systems analysis and design***- The process of developing and maintaining an information system.
12. ***Interface***-Point of contact where a system meets its environment or where subsystems meet each other.
13. ***Interrelated-*** Dependence of one part of the system on one or more other system parts.
14. ***Joint application design (JAD)***- A structured process in which users, managers, and analysts work together for several days in a series of intensive meetings to specify or review system requirements.
15. ***Modularity***- Dividing a system up into chunks or modules of a relatively uniform size.
16. ***Participatory design***- A systems development approach that originated in northern Europe, in which users and the improvement of their work lives are the central focus.
17. ***Prototyping***- Building a scaled-down version of the desired information system.
18. ***Purpose***- The overall goal or function of a system.
19. ***Rapid application Development***- Systems development methodology created to radically decrease the time needed to design and implement information systems.
20. ***Repository***- A centralized database that contains all diagrams, forms and report definitions, data structures, data definitions, process flows and logic, and definitions of other organizational and system components; it provides a set of mechanisms and structures to achieve seamless data-to-tool and data-to-data integration.
21. ***System***- A group of interrelated procedures used for a business function, with an identifiable boundary, working together for some purpose.
22. ***Systems analysis***- During this phase, the analyst thoroughly studies the organization's current procedures and the information systems used to perform tasks such as general ledger, shipping, order entry, machine scheduling, and payroll.

23. ***Systems analyst***- The organizational role most responsible for the analysis and design of information systems.
24. ***Systems design***- Phase of the SDLC in which the system chosen for development in systems analysis is first described independently of any computer platform (logical design), and is then transformed into technology-specific details (physical design) from which all programming and system construction can be accomplished.
25. ***Systems development life cycle***- The series of steps used to mark the phases of development for an information system.
26. ***Systems development methodology***- A standard process followed in an organization to conduct all the steps necessary to analyze, design, implement, and maintain information systems.
27. ***Systems implementation and operation***- During systems implementation and operation, you turn system specifications into a working system that is tested and then put into use. Implementation includes coding, testing, and installation
28. ***Systems planning and selection***- The first phase of the SDLC, in which an organization's total information system needs are analyzed and arranged, and in which a potential information systems project is identified and an argument for continuing or not continuing with the project is presented.

WEEK 2

Sample Gantt Chart



MIS 3373

Systems Analysis & Design Theory Class Project

SIMPLE GANTT CHART

Project Start: Monday, January 27, 2025

Team Number: 6

TASK	ASSIGNED TO	PROGRESS	START	END
Planning				
Project Kickoff Planning	Chisanshi	0%	1/27/25	2/3/25
Project Kickoff Planning	Josephine	0%	1/27/25	2/3/25
Project Kickoff Planning	Kaleb	0%	1/27/25	2/3/25
Project Kickoff Planning	Kensington	0%	1/27/25	2/3/25
Analysis				
Requirements Analysis	Chisanshi	0%	2/5/25	2/17/25
Feasibility Analysis	Josephine	0%	2/5/25	2/17/25
Feasibility Analysis	Kaleb	0%	2/5/25	2/17/25
Requirements Analysis	Kensington	0%	2/5/25	2/17/25
Design				
System Design	Chisanshi	0%	2/19/25	3/12/25
Mock-up Development	Kaleb	0%	2/19/25	3/12/25
Mock-up Development	Josephine	0%	2/19/25	3/12/25
System Design	Kensington	0%	2/19/25	3/12/25
Implementation				
Documentation& Present	Chisanshi	0%	3/24/25	4/18/25
Project Reflection	Kaleb	0%	3/24/25	4/18/25
Final Submission	Josephine	0%	3/24/25	4/18/25
Testing and Risk Mgt	Kensington	0%	3/24/25	4/18/25

Insert new rows ABOVE this one

PERT Formula

$$ET = \frac{o + 4r + p}{6}$$

Where,

ET = expected time for the completion for an activity
o = optimistic completion time for an activity
r = realistic completion time for an activity
p = pessimistic completion time for an activity

Sample Calculation

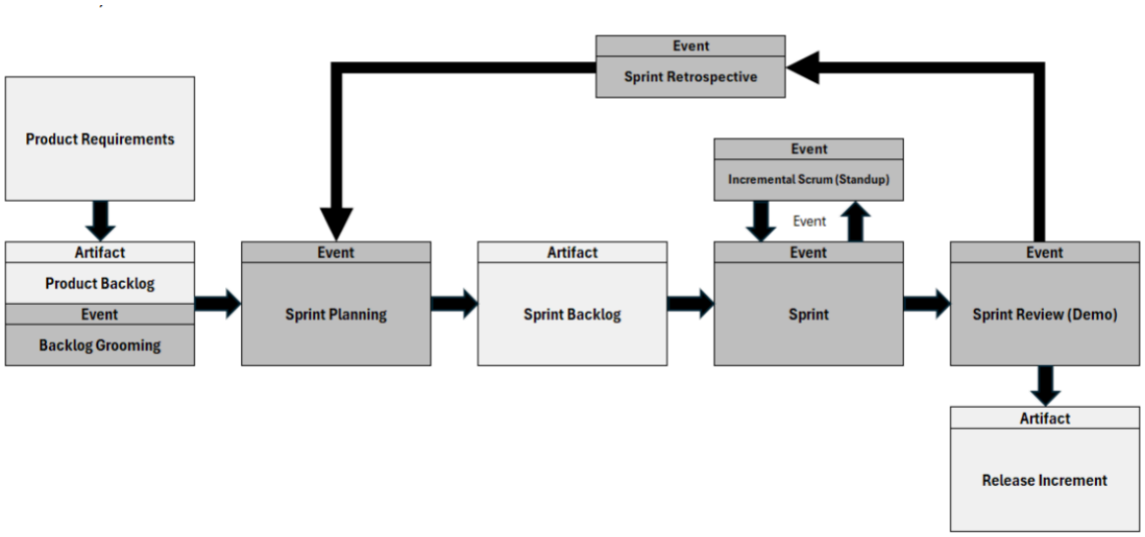
O = Optimistic Time = 10 Days
R = Realistic Time (Best Guess) = 14 Days
P = Pessimistic Time = 12 Days
ET = (O + 4R + P) / 6
ET = (10 + (4*14) + 12) / 6
ET = 13 days

Key Terms and Definitions from Chapter 3

1. ***COCOMO***- Constructive Cost Model, A method for estimating a software project's size and cost.
2. ***Critical path*** - The shortest time in which a project can be completed.
3. ***Critical path scheduling***- A scheduling technique in which the order and duration of a sequence of task activities directly affect the completion date of a project.
4. ***Deliverable***- An end product in a phase of the SDLC.
5. ***Feasibility study***- Determines whether the information system makes sense for the organization from an economic and operational standpoint.
6. ***Gantt chart***- a project management tool, a horizontal bar chart, that visually displays a project's timeline and tasks, their durations, dependencies, and milestones, aiding in planning, scheduling, and tracking progress
7. ***Network diagram***- A diagram that depicts project tasks and their interrelationships
8. ***PERT (program evaluation review technique)***- A technique that uses optimistic, pessimistic, and realistic time estimates to calculate the expected time for a particular task.
9. ***Project***- A planned undertaking of related activities, having a beginning and an end, to reach an objective.
10. ***Project charter***- A short, high-level document prepared for both internal and external stakeholders to formally announce the establishment of the project and to briefly describe its objective, key assumptions, and stakeholders.
11. ***Project closedown***- The final phase of the project management process, which focuses on bringing a project to an end.
12. ***Project execution***- The third phase of the project management process, in which the plans created in the prior phases (project initiation and planning) are put into action.
13. ***Project initiation***- The first phase of the project management process in which activities are performed to assess the size, scope, and complexity of the project and to establish procedures to support later project activities.
14. ***Project management***- A controlled process of initiating, planning, executing, and closing down a project.
15. ***Project manager***- A systems analyst with a diverse set of skills—management, leadership, technical, conflict management, and customer relationship—who is responsible for initiating, planning, executing, and closing down a project.
16. ***Project planning***- The second phase of the project management process, which focuses on defining clear, discrete activities and the work needed to complete each activity within a single project.
17. ***Project workbook***- An online or hard-copy repository, for all project correspondence, inputs, outputs, deliverables, procedures, and standards, that is used for performing project audits, orienting new team members, communicating with management and customers, identifying future projects, and performing postproject reviews.
18. ***Resources***- Any person, group of people, piece of equipment, or material used in accomplishing an activity.
19. ***Slack time***- The amount of time that an activity can be delayed without delaying the project.
20. ***Work breakdown structure (WBS)***- The process of dividing the project into manageable tasks and logically ordering them to ensure a smooth evolution between tasks.

WEEK 3

Scrum Process



SCRUM Roles

Product Owner: Responsible for maintaining the product backlog, prioritizing user stories, and ensuring the team works on the most valuable features.

Scrum Master*:* Facilitates the Scrum process, helps the team follow Scrum practices, and removes any obstacles that may hinder the team's progress.

Development Team: The Development Team is a cross-functional team responsible for designing, developing, testing, and delivering the product increment in each sprint.

SCRUM Artifacts

Artifacts are key components that provide transparency and help the Scrum Team to manage work effectively throughout the project. They represent essential information that is created, refined, and shared during the course of the Scrum process. There are three primary Scrum artifacts:

Product Backlog: The prioritized list of user stories, features, and tasks that need to be completed. The Product Owner maintains and continuously refines the backlog.

Sprint Backlog: The set of user stories and tasks selected from the product backlog for the current sprint. The Development Team owns the sprint backlog.

Increment: The sum of all the completed user stories and features at the end of a sprint, resulting in a potentially shippable product.

Product Requirements v. Product Backlog

Product Requirements: High-level needs or objectives for the product. They describe what the product should do, often from the perspective of stakeholders or end users. Requirements are typically broader and can be both functional (what the product should do) and non-functional (how the product should perform, e.g., security, speed).

Product Backlog: A prioritized list of work that needs to be done to create or enhance the product. It is a dynamic and living document managed by the Product Owner. The backlog consists of backlog items, which could include user stories, features, technical tasks, bug fixes, or improvements. The product backlog represents the work needed to fulfill the product requirements.

SCRUM Events

Sprint Planning: At the beginning of each sprint, the Product Owner and the Development Team collaborate to select items from the product backlog and define the sprint goal.

Sprint: The time-boxed period in which work is done.

Periodic Stand Up (Scrum): A brief daily meeting for the Development Team to synchronize, share progress, and discuss any obstacles. Each team member answers three questions: What did I do yesterday? What am I doing today? Are there any obstacles?

Sprint Review (Demo): At the end of each sprint, the Development Team presents the completed work to stakeholders, receives feedback, and discusses what was accomplished during the sprint.

Sprint Retrospective: Also held at the end of each sprint, the retrospective is a reflection on the sprint's process, where the team discusses what went well, what could be improved, and plans actions for improvement in the next sprint.

Backlog Grooming: Regular sessions where the Product Owner and Development Team review and prioritize items in the product backlog, adding details and estimates as needed.

User Story

A user story is a concise and informal description of a feature or functionality from the perspective of an end user or customer. It is a common technique used in agile software development methodologies, such as Scrum, to capture and communicate requirements in a more user-centered and understandable way. User stories are a way to express the "who," ,"what," and "why" of a particular piece of functionality.

Components of a User Story

Role: This is the user or persona for whom the feature is being developed. It describes the person or entity that will interact with the system.
Action: This describes what the user wants to achieve or do with the system. It outlines the specific task or action that the user wants to perform.

Benefit: This explains the value or benefit that the user will gain from completing the action. It highlights why the feature is important and how it contributes to the overall goals of the project.

Preparing for a Sprint

1. **Backlog Refinement (Grooming):** An ongoing process where the Scrum Team collaboratively reviews, refines, and prepares items in the Product Backlog.

Key Aspects:
Reviewing and Clarifying User Stories: The Product Owner and the Development Team review the items (user stories, features, tasks, etc.) in the product backlog. The aim is to ensure that these items are well-understood and their requirements are clear. Any uncertainties or ambiguities are discussed and clarified.

Breaking Down Items: If a backlog item is too large or complex, the team might break it down into smaller, manageable tasks or sub-stories. Breaking down items improves understanding, facilitates estimation, and allows for more accurate planning.

Collaboration: Backlog grooming is a collaborative effort involving the Product Owner,Scrum Master, and the Development Team. Effective communication and discussion help ensure that everyone has a clear understanding of the backlog items.

Estimation: The development team often estimates the effort required for each backlog item. This could be done using techniques like story points or relative sizing. Estimation helps the team and the Product Owner understand the scope of each item and make informed prioritization decisions.

Updating Details: The team might add additional information, details, and acceptance criteria to backlog items to ensure a shared understanding of what is expected. This helps prevent misunderstandings during development.

Frequency: Backlog grooming is not a separate Scrum event but rather a continuous process. It is typically conducted as needed throughout the sprint to ensure that the upcoming backlog items are well-prepared for the next sprint planning meeting.

Prioritization: The Product Owner may adjust the priorities of backlog items based on changing marked needs, customer feedback, or other business considerations. Backlog grooming provides an opportunity to reassess and reprioritize items as needed.

Removing or Archiving Items: Backlog grooming can also involve identifying and removing items that are no longer relevant or necessary. This helps keep the backlog focused on the most valuable and relevant work.

2. **Story Points:** A relative measure used in agile methodologies, particularly in scrum, to estimate the effort, complexity, and size of user stories or backlog items. User stories are short descriptions of a feature or functionality from an end-user perspective, written in a way that captures the value and requirements of that feature. Rather than providing precise time-based estimates (like hours or days), agile teams use story points to indicate the perceived effort required to complete a user story. Story points are a way to measure the relative difficulty of one user story compared to another, rather than trying to pinpoint an exact amount of time the work will take.

How do they work?

Relative Estimation: Teams compare user stories against a reference story (sometimes called a "baseline" or "anchor" story) that is given a certain number of story points. The team then estimates other stories based on how they perceive the effort required relative to the reference story.

Fibonacci Scale: Story points are often assigned using a scale based on the Fibonacci sequence (1,2,3,5,8,13,21,etc.). It is a sequence of numbers where each number is the sum of the two preceding ones. This scale reflects the inherent uncertainty in estimating complex tasks. As the points increase, the uncertainty and complexity also increase.

Team Collaboration: Estimating story points is a collaborative effort involving the entire development team, including developers, testers, and other relevant roles. Different team members bring their perspectives to the estimation process, which can lead to more accurate assessments.

Factors Considered: Teams consider factors such as the complexity of the work, potential technical challenges, dependencies, risks, and the overall effort required to complete the story. Story points encompass various aspects of the work, not just the time it takes to code.

Velocity: Over time, a team's historical performance is measured in terms of the number of story points they complete per spring, which is referred to as "velocity." Velocity helps the team plan how much work they can commit to in future sprints based on their past performance.

Benefits
- They focus on relative sizing rather than trying to predict the exact time.
- They accommodate the inherent uncertainty in projects.
- They encourage collaboration and shared understanding within the team.
- They help in capacity planning and establishing realistic

What is a Story Point?
- The point value assigned to a task defines the anticipated level of difficulty.
- It is important that points are based on a "starting point."
- Example story point matrix: notice that 13 was defined as taking one month!
- Each team member should take a similar amount of points.

Baseline
Determining the baseline or starting point for story points is a critical aspect of using the story point estimation technique in agile methodologies like Scrum.

- ***Select a Reference Story***: Choose a user story that is of moderate complexity and represents a typical piece of work for your team. This reference story will serve as the baseline for your story point scale. It's important that the team has a shared understanding of this story's complexity.

- ***Assign Story Points***: Assign a story point value to the reference story. This value will act as your baseline or anchor point. Some teams start with a value of 1, while others might use a value of 2, 3, or another number that feels appropriate for their context.

- ***Relative Comparison:*** Once you have your baseline story point value, compare other user stories against this reference story. Ask the question: "Is this new story more, less,or equally complex compared to our reference story?"

- ***Use the Fibonacci Scale:*** Use the Fibonacci sequence (1, 2, 3, 5, 8, 13, etc.) to assign story point values to new user stories based on their relative complexity compared to the reference story. For example, if a new story is somewhat more complex than the reference story, you might assign it a higher story point value like 3, 5, or 8.

- ***Team Discussion and Consensus:*** Involve the entire development team in the estimation process. Encourage open discussions where team members share their perspectives on the complexity and effort involved in each story. Reach a consensus on the story point value by considering different viewpoints.

- ***Iterative Refinement:*** As your team gains experience and becomes more familiar with the story point estimation process, you may find that your initial baseline needs adjustment. This is normal and expected. Over time, your understanding of story points will improve, and you'll refine your scale accordingly.

- ***Avoid Paralysis by Analysis:*** Remember that story points are meant to be a tool for relative estimation, not an exact science. The goal is to quickly and collaboratively estimate the relative complexity of user stories, not to spend excessive time on precise calculations.
Story point scales and practices can vary from one team to another based on the team's unique context, experience level, and work dynamics. The key is to establish a baseline that the team can use consistently for estimating user stories and to continuously improve the estimation process over time.

3. **Sprint Planning:** A collaborative and time-boxed activity that ensures alignment between the development work and the product's goals. It helps the team prioritize and plan their work effectively, promoting transparency and a shared understanding of the work to be accomplished.

How It Works
***Attendees*:** The spring planning meeting involves the entire Scrum Team, which includes the product owner, scrum master, and the development team.

Part 1: What to Build: The product owner presents the prioritized items from the product backlog that they believe should be worked on in the upcoming sprint. These items can include user stories, features, bug fixes, and technical tasks.
Clarification and Discussion: The development team has an opportunity to ask questions and seek clarification about the requirements and expectations of each item. This helps ensure a shared understanding of what need to be built.

Part 2: How to Build: Based on the information provided, the development team discusses how they will implement the selected items. They identify the tasks or subtasks required to complete each user story or backlog item.
Capacity Planning: The team considers their capacity – how much work they can handle during the sprint. This involves factoring in holidays, meetings, and other potential interruptions that might affect their availability.

Commitment: Based on the discussions, estimations, and capacity considerations, the development team and product owner work together to decide which items will be taken into the sprint backlog. This is the commitment the team makes for the sprint.

4. **Sprint Review (Demo):** Inspect the work completed during the sprint and gather feedback from stakeholders, including the Product Owner, end users, customers, and other relevant parties. The sprint review is a valuable opportunity for the scrum team to demonstrate what has been accomplished and gather insights that can inform future work.

How It Works
Attendees: The sprint review involves the entire scrum team, including the product owner, scrum master, and the development team. Additionally, stakeholders such as end-users, customers, and anyone else interested in the product are invited to attend.
Demo of Completed Work: The development team showcases the work they have completed during the sprint. This typically involves demonstrating new features, enhancements, or bug fixes that were part of the sprint backlog.
Inspection: Stakeholders have the opportunity to see the working product increment and interact with it. The goal is to assess whether the work meets the requirements and expectations set out for the sprint.
Feedback and Discussion: After the demonstration, the product owner and stakeholders provide feedback on the demonstrated features. This feedback is crucial for the team to understand if they are on the right track and to make any necessary adjustments or improvements.
Adaptation: Based on the feedback received and the insights gained during the sprint review, the scrum team and the product owner may decide to adjust the product backlog or change priorities for future sprints.
Backlog Refinement: The feedback gathered during the sprint review can lead to the identification of new user stories, refinements to existing stories, or changes to the overall product roadmap.
Continuous Improvement: The sprint review is a platform for reflection and learning. The team discusses what went well during the sprint and identifies areas for improvement in their processes, collaboration, and product quality.

5. **Sprint Retrospective:** Provides the scrum team with a dedicated time to reflect on the sprint that just concluded. It is an opportunity for the team to discuss what went well, what could be improved, and to make actionable commitments to enhance their processes in the future. It is a cornerstone of the agile principle of continuous improvement. It encourages transparency, collaboration, and a culture of learning within the team. By taking time to reflect on their work and processes, the team can make meaningful adjustments to enhance their efficiency and deliver higher value with each successive sprint.

How It Works

Attendees: The Sprint Retrospective involves the Scrum Team members, including the Product Owner, Scrum Master, and the Development Team. It's a private event where the team can openly discuss their experiences.

Reflect on the Sprint: The team reflects on the completed sprint, considering aspects like the work completed, collaboration, communication, and any challenges encountered.

Positive Feedback: The team starts by discussing and highlighting the positive aspects of the sprint. This could include accomplishments, successful teamwork, and effective practices that contributed to the sprint's success.

Areas for Improvement: The team then shifts their focus to identifying areas that can be improved. These could be process-related, communication-related, technical challenges, or any other aspects that could be optimized.

Generate Insights: During this phase, the team openly shares their perspectives on what worked well and what could be better. This discussion is meant to be constructive and solution-oriented.

Actionable Items: Based on the discussion, the team identifies specific action items or improvement opportunities that they believe will lead to a more effective and efficient workflow in the upcoming sprints.

Ownership: The team members take ownership of the identified action items. These could involve changes in processes, communication practices, collaboration strategies, or technical approaches.

Prioritization: The team might decide to prioritize the identified improvements and determine which ones are feasible to address in the immediate future.

Implement and Adapt: The action items generated during the retrospective are used to drive improvements in subsequent sprints. Teams should actively experiment with new approaches and continuously adapt their practices.

Key Terms Appendix B

1. ***Agile Methodologies***- A family of development methodologies characterized by short iterative cycles and extensive testing; active involvement of users for establishing, prioritizing, and verifying requirements; and a focus on small teams of talented, experienced programmers.
2. ***Refactoring***- Making a program simpler after adding a new feature.
3. ***Simple Design***- Creating uncomplicated software and software components that work to solve the current problem rather than creating complicated software designed for a future that may not come.

WEEK 4

Build vs Buy Questionnaire

Software Build vs. Buy Questionnaire
This questionnaire aids in the evaluation of whether to build custom software in-house or purchase an off-the-shelf solution. There are many factors to consider when evaluating potential software solutions. This questionnaire provides a framework of these factors to help guide in decision making.ch

Business Needs & Goals

1. **Does the software need to be tailored to meet unique business processes or industry-specific needs?**

   Yes (Build) - If our organization has specific internship placement workflows or needs custom features, building the tool makes sure it aligns perfectly with our processes.
   No (Buy)

2. **How critical is the software to your core business operations or competitive advantage?**

   Highly critical (Build)
   Moderately critical (Buy) - If the software is important but not the main driver of the business or competitive edge, buying an off-the-shelf solution is a practical choice that gets the job done without the complexity of custom development.

3. **Is this a long-term solution or a temporary fix?**

   Long-term (Build) - If this tool will be a key part of our operations for years and needs to grow and adapt alongside our business, building a custom solution ensures it stays flexible and meets your evolving needs.
   Temporary (Buy)

4. **Can existing off-the-shelf software meet 80–90% of your requirements?**

   No (Build)
   Yes (Buy) - If an off-the-shelf solution already covers most of our needs with minimal adjustments, buying saves time and effort while still providing a functional and reliable tool.

Cost Considerations

5. **Do you have the budget for upfront custom development costs?**

   Yes (Build)
   No (Buy) - If upfront development costs are a concern and we need a more affordable solution that works out of the box, buying off-the-shelf software is the better choice.

6. **What is the total cost of ownership (TCO) over time (including development, licensing, support, and maintenance)?**

   Higher TCO acceptable (Build)
   Lower TCO preferred (Buy) - If keeping costs manageable over time is a priority and we'd rather avoid these expenses, buying an off-the-shelf solution is a more budget-friendly choice.

7. **Will the off-the-shelf solution require significant customization (and additional costs) to fit your needs?**

   Yes (Build)
   No (Buy) - If an off-the-shelf solution already meets our needs with little to no customization required, buying would be the smarter choice—it saves time, reduces costs, and lets us get up and running quicker.

Time to Market

8. **How soon do you need the solution implemented?**

   Flexible timeline (Build)
   Immediate need (Buy) - If we need the software up and running as soon as possible, buying a ready-made solution is the fastest way to get started without the long development process.

9. **Can your team manage the project timeline for building software without impacting other priorities?**

   Yes (Build) - If our team has the capacity to handle the development timeline without disrupting any other key priorities, building a custom solution gives us more control and flexibility in the long run.
   No (Buy)-

Internal Resources

10. **Does your team have the technical expertise and resources to develop and maintain custom software?**

Yes (Build)

No (Buy) - If we don't have the technical expertise or resources to build and maintain custom software, buying a ready-made solution ensures we get the functionality needed without the hassle of development and ongoing upkeep.

11. **Can your team handle ongoing maintenance, updates, and support for the custom solution?**

Yes (Build) - If we have the skills and capacity to manage maintenance, updates, and support, a custom solution gives us full control and the flexibility to make improvements as needed.

No (Buy)

Integration & Scalability

12. **How well does the off-the-shelf solution integrate with your current systems and workflows?**

Poorly (Build)

Well (Buy) - If the off-the-shelf solution integrates smoothly with our existing systems and workflows, buying is the simpler and more efficient choice.

13. **Will the software need to scale significantly as your business grows?**

Yes (Build)

No (Buy) - If we don’t anticipate major growth or the need for extensive new features, buying an off-the-shelf solution is a more cost-effective choice without the hassle of custom development.

14. **Will customization of an off-the-shelf product negatively impact scalability?**

Yes (Build)

No (Buy) - If an off-the-shelf solution can scale with our business without major limitations, buying is the smarter choice since it saves time and effort while still allowing for growth.

Risk Management

15. **Are you concerned about vendor lock-in with an off-the-shelf solution?**

Yes (Build)

No (Buy) - If vendor lock-in isn't a major concern and we're comfortable using a reliable third-party solution, buying off-the-shelf software allows us to get up and running quickly without having to build and maintain a custom tool.

16. **Do you require full control over the software, including source code and intellectual property?**

    Yes (Build)
    No (Buy) - If having full control over the software isn't a priority, buying a ready-made solution is the easier route—it saves us time, reduces maintenance responsibilities, and lets us focus on using the tool rather than managing its development.

17. **How much risk can your organization handle in terms of potential delays, bugs, or cost overruns in development?**

    High risk tolerance (Build)
    Low risk tolerance (Buy) - If our organization wants to avoid these risks, buying a ready-made solution is a more reliable choice.

Vendor Evaluation (if Buying)

18. **Does the vendor have a proven track record and solid customer support?**

    Yes (Buy) - If the vendor has a strong reputation and reliable customer support, buying their solution gives us a struggle-free option with expert assistance when we need it.
    No (Reevaluate)

19. **Are the vendor's pricing, terms, and SLA (Service Level Agreement) reasonable and transparent?**

    Yes (Buy) - If the vendor offers fair pricing, clear terms, and a reliable service agreement, buying their solution is an easy way to get up and running without the complexities of building from scratch.
    No (Reevaluate)

20. **Does the off-the-shelf solution provide regular updates and enhancements to meet evolving needs?**

    Yes (Buy) - If the off-the-shelf solution is regularly updated with new features and improvements, it can adapt to our evolving needs without having to manage development ourselves.
    No (Reevaluate)

Decision Framework

- **If most of your answers favor Build:** Consider developing a custom solution to meet your unique needs and maintain control.

- **If most of your answers favor Buy:** Opt for an off-the-shelf solution to save time, resources, and costs.

- **If answers are mixed:** Conduct a deeper analysis, such as a cost-benefit analysis, or consider hybrid options (buying a solution and customizing it)

Chapter 2 Key Terms

1. ***Cloud Computing***- The provision of computing resources, including applications, over the Internet, so customers do not have to invest in the computing infrastructure needed to run and maintain the resources.
2. ***Enterprise resource planning-*** A system that integrates individual traditional business functions into a series of modules so that a single transaction occurs seamlessly within a single information system rather than several separate systems.
3. ***Outsourcing***-The practice of turning over responsibility for some or all of an organization's information systems applications and operations to an outside firm.
4. ***Request for proposal (RFP)***- A document provided to vendors to ask them to propose hardware and system software that will meet the requirements of a new system.
5. ***Reuse-*** The use of previously written software resources, especially objects and components, in new applications.

WEEK 5

Assessing Project Feasibility

Project feasibility factors fall into six categories:

1. Economic feasibility- identifying the financial benefits and costs associated with the development project, also known as cost-benefit analysis

    - ***Net present value of all benefits*** *= Net economic benefit x discount rate= PV of Benefits*
    - ***Net present value of all costs*** *= Recurring costs x discount rate*
    - ***Overall Net present value*** *= NPV of all benefits + NPV of all costs*

    - ***Return on investment (ROI)-*** *the ratio of the net cash receipts of the project divided by the cash outlays of the project. Trade-off analysis can be made among projects competing for investment by comparing their representative ROI ratios*

      ROI= Overall NPV/ NPV of all costs

    - ***Break-even analysis-*** *BEA finds the amount of time required for the cumulative cash flow from a project to equal its initial and ongoing investment.*

$$\text{Break-Even Ratio} = \frac{\text{Yearly NPV Cash Flow} - \text{Overall NPV Cash Flow}}{\text{Yearly NPV Cash Flow}}$$

2. Operational feasibility- The process of examining the likelihood that the project will attain its desired objectives.
3. Technical feasibility- Understanding the development organization's ability to construct the proposed system.
4. Schedule feasibility- Considers the likelihood that all potential time frames and completion-date schedules can be met and that meeting these dates will be sufficient for dealing with the needs of the organization
5. Legal and contractual feasibility- Requires that you gain an understanding of any potential legal and contractual ramifications due to the construction of the system
6. Political feasibility- Understanding how key stakeholders within the organization view the proposed system

Operational Feasibility Assessment Framework Example: Internship Acquisition Tool

1. Stakeholder Analysis

Identify the key users and assess their willingness to adopt the system. For the Role in the System, identify each Stakeholder group's role in the system. In the Feasibility Considerations, list the questions that come to mind regarding the stakeholder group identified. See the examples below (which you can use). Simply complete the Stakeholder Analysis chart below.

Stakeholder Group	Role in System	Feasibility Considerations (Questions to Address)
Students	Users searching and applying for internships.	Will students actively use the system over external job boards?
Employers	Posting internship opportunities and reviewing applicants.	Will employers be willing to post internships and engage with student applicants?
University Career Services	Managing internship listings, assisting students, and employer outreach.	Will Career Services staff have the necessary time and resources to support the system?
IT Support Team	Maintaining system functionality, security, and troubleshooting.	Can the IT team effectively manage system updates and resolve technical issues?

2. Business Process Alignment

Analyze how the system integrates with existing university processes. For the Current State, identify the Current State of each Process Area. In the Impact of New System, list the questions that come to mind regarding the process area identified. See the examples below (which you can use). Simply complete the remaining Business Process Alignment chart below.

Process Area	Current State	Impact of New System (Questions to Address)
Internship Search & Application	Manual or using multiple platforms (Handshake, LinkedIn).	Will students prefer a centralized system?
Employer Engagement	Employers manually post opportunities on different platforms or contact career services directly.	Will employers adopt the new system for posting internships and reviewing applicants?

Student Career Counseling	Career advisors track student progress and opportunities manually or via emails.	Will the system help career advisors streamline student guidance and tracking?
Reporting & Metrics	Data is collected manually or through multiple disconnected systems.	Will the system provide accurate and real-time reports for decision-making?

3. User Experience & Adoption

Assess system usability and acceptance. For the Consideration, identify the consideration(s) regarding the Factor listed. See the example below (which you can use). Simply complete the remaining User Experience & Adoption chart below.

Factor	Consideration Questions to Address)
Ease of Use	s the system intuitive for students and employers?
Training Needs	Will students, employers, and career advisors require training to use the system effectively? What resources will be needed?
Resistance to Change	How can the university encourage adoption and minimize reluctance from takeholders?
Mobile Compatibility	Will the system work seamlessly on mobile devices for easy access by students nd employers?

4. Resource Availability

Evaluate whether the university has the resources to support the system. For the Availability, identify what must exist for each Resource Type. For the Concerns, list the questions that come to mind regarding the Resource Type and Availability identified. See the example below (which you can use). Simply complete the remaining Resource Availability chart below.

Resource Type	Availability	Concerns Questions to Address)
IT Support	University IT team.	Do they have bandwidth for maintenance and updates?
Financial	University budget allocation or grant funding.	s there a budget for software development, licensing, nd ongoing support? Who approves the funding?

Human Resources	Faculty supervisors, student developers, and university staff.	Are there enough qualified personnel to develop and maintain the system? Will student interns receive training and support?

5. Security & Compliance

Ensure legal and technical compliance. For the Compliance Check, identify what must be considered when addressing the Concern. See the example below (which you can use). Simply complete the remaining Security & Compliance chart below.

Concern	Compliance Check (Questions to Consider)
Data Privacy	Does it comply with FERPA (Family Educational Rights and Privacy Act)?
Security	Are user authentication and access controls in place? Is data encryption used for sensitive information? How is the system protected against cyber threats (e.g., firewalls, antivirus, secure coding practices)?
Accessibility	Does the software comply with WCAG (Web Content Accessibility Guidelines) to ensure usability for individuals with disabilities? Is it compatible with screen readers and assistive technologies?

6. Risk Assessment & Mitigation

Identify and address potential risks. For the Probability, assign a value of Low, Medium, or High that represents the probability of the Risk occurring. For the Impact, assign a value of Low, Medium, or High that represents the Impact of the Risk occurring. For the Mitigation Strategy, describe how the Risk will be minimized. See the example below (which you can use). Simply complete the remaining Risk Assessment & Mitigation chart below.

Risk	Probability (Low, Medium, High)	Impact (Low, Medium, High)	Mitigation Strategy (How is the Risk Minimized)
Low Student Adoption	Medium	High	User incentives, ease of use, integration with university accounts.
Employer Participation	Medium	High	Engage employers early, offer incentives, streamline job posting process, and provide strong support.
System Downtime	Medium	High	Regular system maintenance, robust hosting infrastructure, backup and recovery plans.
Compliance Issues	Low	High	Conduct regular audits, ensure adherence to FERPA and accessibility standards, train staff on compliance requirements.

Baseline Project Plan (BPP)

One of the major outcomes and deliverables from the project initiation and planning phase. It contains the best estimate of the project's scope, benefits, costs, risks, and resource requirements.

- Introduction
- System description
- Feasibility assessment
- Management issues

Example Baseline Project Plan Report

BASELINE PROJECT PLAN REPORT
1.0 Introduction A. Project Overview—Provides an executive summary that specifies the project's scope, feasibility, justification, resource requirements, and schedules. Additionally, a brief statement of the problem, the environment in which the system is to be implemented, and constraints that affect the project are provided. B. Recommendation—Provides a summary of important findings from the planning process and recommendations for subsequent activities.
2.0 System Description A. Alternatives—Provides a brief presentation of alternative system configurations. B. System Description—Provides a description of the selected configuration and a narrative of input information, tasks performed, and resultant information.
3.0 Feasibility Assessment A. Economic Analysis—Provides an economic justification for the system using cost-benefit analysis. B. Technical Analysis—Provides a discussion of relevant technical risk factors and an overall risk rating of the project. C. Operational Analysis—Provides an analysis of how the proposed system solves business problems or takes advantage of business opportunities in addition to an assessment of how current day-to-day activities will be changed by the system. D. Legal and Contractual Analysis—Provides a description of any legal or contractual risks related to the project (e.g., copyright or nondisclosure issues, data capture or transferring, and so on). E. Political Analysis—Provides a description of how key stakeholders within the organization view the proposed system. F. Schedules, Timeline, and Resource Analysis—Provides a description of potential time frame and completion-date scenarios using various resource allocation schemes.
4.0 Management Issues A. Team Configuration and Management—Provides a description of the team member roles and reporting relationships. B. Communication Plan—Provides a description of the communication procedures to be followed by management, team members, and the customer. C. Project Standards and Procedures—Provides a description of how deliverables will be evaluated and accepted by the customer. D. Other Project-Specific Topics—Provides a description of any other relevant issues related to the project uncovered during planning.

Project Scope Statement

A document prepared for the customer that describes what the project will deliver and outlines generally at a high level all work required to complete the project.

It generally includes the following information

- Header
- General Project Information
- Problem/ Opportunity Statement
- Project Objectives
- Project Description
- Business benefits
- Project deliverables
- Estimated Project Duration

Project scope statement example:

Pine Valley Furniture ***Project Scope Statement***	Prepared by: Jim Woo Date: September 20, 2015
General Project Information **Project Name:** Customer Tracking System **Sponsor:** Jackie Judson, VP Marketing **Project Manager:** Jim Woo	
Problem/Opportunity Statement: Sales growth has outpaced the marketing department's ability to track and forecast customer buying trends accurately. An improved method for performing this process must be found in order to reach company objectives.	
Project Objectives: To enable the marketing department to track and forecast customer buying patterns accurately in order to better serve customers with the best mix of products. This will also enable PVF to identify the proper application of production and material resources.	
Project Description: A new information system will be constructed that will collect all customer purchasing activity, support display and reporting of sales information, aggregate data, and show trends in order to assist marketing personnel in understanding dynamic market conditions. The project will follow PVF's systems development life cycle.	
Business Benefits: Improved understanding of customer buying patterns Improved utilization of marketing and sales personnel Improved utilization of production and materials	
Project Deliverables: Customer tracking system analysis and design Customer tracking system programs Customer tracking documentation Training procedures	
Estimated Project Duration: 5 months	

Key Terms and Definitions Chapter 4

1. ***Baseline project plan***-One of the major outcomes and deliverables from the project initiation and planning phase. It contains the best estimate of the project's scope, benefits, costs, risks, and resource requirements.
2. ***Break-even analysis-*** A type of cost-benefit analysis to identify at what point (if ever) benefits equal costs.
3. ***Business case-*** A written report that outlines the justification for an information system. The report highlights economic benefits and costs and the technical and organizational feasibility of the proposed system.
4. ***Discount rate***-The interest rate used to compute the present value of future cash flows.
5. ***Economic feasibility-*** A process of identifying the financial benefits and costs associated with a development project.
6. ***Electronic commerce-*** Internet-based communication and other technologies that support day-to-day business activities.
7. ***Incremental commitment***-A strategy in systems analysis and design in which the project is reviewed after each phase, and continuation of the project is rejustified in each of these reviews.
8. ***Intangible benefit-*** A benefit derived from the creation of an information system that cannot be easily measured in dollars or with certainty.
9. ***Intangible cost-*** an expense that can't be quantified or expressed as a monetary value
10. ***Legal and contractual feasibility***- The process of assessing potential legal and contractual ramifications due to the construction of a system.
11. ***One-time cost-*** A cost associated with project initiation and development, or system start-up.
12. ***Operational feasibility***-The process of assessing the degree to which a proposed system solves business problems or takes advantage of business opportunities.
13. ***Political feasibility***- The process of evaluating how key stakeholders within the organization view the proposed system.
14. ***Present value***-The current value of a future cash flow.
15. ***Project scope statement-*** A document prepared for the customer that describes what the project will deliver and outlines generally at a high level all work required to complete the project.
16. ***Recurring cost-***A cost resulting from the ongoing evolution and use of the system.
17. ***Schedule feasibility***-The process of assessing the degree to which the potential time frame and completion dates for all major activities within a project meet organizational deadlines and constraints for effecting change.
18. ***Tangible benefit-*** A benefit derived from the creation of an information system that can be measured in dollars and with certainty.
19. ***Tangible cost-*** A cost associated with an information system that can be easily measured in dollars and with certainty.
20. ***Technical feasibility***-The process of assessing the development organization's ability to construct a proposed system.
21. ***Time value of money (TVM)***- The process of comparing present cash outlays to future expected returns.

22. ***Walkthrough***- A peer-group review of any product created during the systems development process; also called a structured walkthrough.

WEEK 6

Types of Deliverables and Specific Deliverables for Requirements Determination

Types of Deliverables	Specific Deliverables
Information collected from conversations with users	Interview transcripts Notes from observations Meeting notes
Existing documents and files	Business mission and strategy statement Sample business forms and reports, and computer displays Procedure manuals Job descriptions Training manuals Flowcharts and documentation of existing systems Consultant reports
Computer-based information	Results from joint application design (JAD) sessions CASE repository contents and reports of existing systems Displays and reports from system prototypes

Project Requirements Document Template

1. Project Overview

Project Name: Internship Locator & Application System (ILAS)
Team Members: [List Names]
Course: MIS 3373 Systems Analysis and Design Theory
Instructor: Professor John T. James

1.1 Purpose

Definition: This section describes the overall goal of the project, outlining why the system is being developed and what problem it aims to solve.
The purpose of the Internship Locator & Application System (ILAS) is to help students find, track, and apply for internships efficiently. The system will streamline the process by providing a centralized platform that connects students with potential employers and offers application tracking and career resources.

1.2 Scope

Definition: This section defines the boundaries of the project, specifying what features and functionalities will be included in the system.
The system will include:
• A student user portal for searching internships, submitting applications, and tracking status.
• An employer portal for posting internships and reviewing applications.
• An administrator dashboard for system management.
• Integration with university career services and external job boards.

2. Functional Requirements

2.1 User Roles

Definition: This section lists the different types of users who will interact with the system and their specific permissions or capabilities.
• Students: Search for internships, apply online, upload resumes, track applications.
• Employers: Post internships, view applications, communicate with applicants.
• Administrators: Manage users, approve job postings, generate reports.

2.2 Core Features

Definition: This section details the key functionalities that the system must have to meet user needs.
• Internship Search: Keyword-based search and filtering by location, industry, company, and date.
• Application Management: Students can apply for internships and track their application progress.
• Employer Dashboard: Manage postings, review applications, and contact applicants.
• Notifications & Alerts: Email or in-app notifications for application updates and deadlines.

• Resume & Profile Upload: Students can create and update profiles and upload resumes.
• Admin Panel: Manage users, moderate job listings, generate reports on user activity.

3. Non-Functional Requirements

3.1 Performance

Definition: This section describes the expected system performance and response times under normal usage conditions.
• The system should handle up to 500 concurrent users.
• Internship search results should load within 3 seconds.

3.2 Security

Definition: This section outlines security measures to protect data and ensure system integrity.
• Secure login with multi-factor authentication (MFA) for administrators.
• Data encryption for stored resumes and sensitive user information.
• Role-based access control to protect employer and student data.

3.3 Usability

Definition: This section highlights usability considerations to ensure a smooth user experience.
• Responsive design for mobile and desktop accessibility.
• Intuitive UI with minimal learning curve.

4. System Architecture

4.1 Technology Stack

Definition: This section lists the software and technologies that will be used to develop the system.
• Frontend: React.js or Angular
• Backend: Node.js with Express or Django
• Database: PostgreSQL or MongoDB
• Hosting: AWS, Azure, or Firebase

4.2 Integration

Definition: This section outlines external systems or services that the project will integrate with.
• University career services portal API
• Third-party job boards (Handshake, LinkedIn, etc.)

5. Constraints & Assumptions

Definition: This section details any constraints that limit project implementation and assumptions made during development.
• Users must have a valid university email to register.
• Employers must verify company legitimacy before posting jobs.
• The system should comply with data privacy laws (FERPA, GDPR if applicable).

6. Milestones & Timeline

Definition: This section provides a structured timeline for completing various project phases.

1. Requirement Gathering & Analysis - Week 1-2
2. System Design & Wireframing - Week 3-5
3. Prototype Development - Week 6-8
4. Testing & QA – Not applicable for this project.
5. Final Implementation & Presentation - Week 9

7. Risks & Mitigation Strategies

Definition: This section identifies potential risks that could impact the project and strategies to mitigate them.

• Risk: Low user adoption → Mitigation: Conduct user feedback sessions.
• Risk: Data security threats → Mitigation: Implement encryption and secure authentication.
• Risk: Scope creep → Mitigation: Define clear project scope and prioritize core features.

8. Conclusion

Definition: This section summarizes the document and reiterates the project's importance.

This document serves as the foundation for the development of ILAS, ensuring all stakeholders understand the project goals, scope, and requirements. Feedback and iterative refinement will be essential to achieving a successful implementation.

Key Terms and DefinitionsChapter 5

1. ***Business process reengineering (BPR)-*** The search for, and implementation of, radical change in business processes to achieve breakthrough improvements in products and services.
2. ***Closed-ended questions-*** Questions in interviews and on questionnaires that ask those responding to choose from among a set of specified responses.
3. ***Disruptive technologies-*** Technologies that enable the breaking of long-held business rules that inhibit organizations from making radical business changes.
4. ***Formal system-*** The official way a system works, as described in organizational documentation.
5. ***Informal system-***The way a system actually works.
6. ***JAD session leader-***The trained individual who plans and leads joint application design sessions.
7. ***Key business processes-***The structured, measured set of activities designed to produce a specific output for a particular customer or market.
8. ***Open-ended questions-*** Questions in interviews and on questionnaires that have no prespecified answers.
9. ***Scribe-*** The person who makes detailed notes of the happenings at a joint application design session.

WEEK 7

16 Rules Governing Data-Flow Diagramming

Rules Governing Data-Flow Diagramming

Process

A. No process can have only outputs. It is making data from nothing (a miracle). If an object has only outputs, then it must be a source.

B. No process can have only inputs (a black hole). If an object has only inputs, then it must be a sink.

C. A process has a verb-phrase label.

Data Store

D. Data cannot move directly from one data store to another data store. Data must be moved by a process.

E. Data cannot move directly from an outside source to a data store. Data must be moved by a process that receives data from the source and places the data into the data store.

F. Data cannot move directly to an outside sink from a data store. Data must be moved by a process.

G. A data store has a noun-phrase label.

Source/Sink

H. Data cannot move directly from a source to a sink. They must be moved by a process if the data are of any concern to our system. Otherwise, the data flow is not shown on the DFD.

I. A source/sink has a noun-phrase label.

Data Flow

J. A data flow has only one direction of flow between symbols. It may flow in both directions between a process and a data store to show a read before an update. The latter is usually indicated, however, by two separate arrows because the read and update usually happen at different times.

K. A fork in a data flow means that exactly the same data go from a common location to two or more different processes, data stores, or sources/sinks (it usually indicates different copies of the same data going to different locations).

L. A join in a data flow means that exactly the same data come from any of two or more different processes, data stores, or sources/sinks to a common location.

M. A data flow cannot go directly back to the same process it leaves. At least one other process must handle the data flow, produce some other data flow, and return the original data flow to the beginning process.

N. A data flow to a data store means update (delete or change).

O. A data flow from a data store means retrieve or use.

P. A data flow has a noun-phrase label. More than one data-flow noun phrase can appear on a single arrow as long as all of the flows on the same arrow move together as one package.

4 Advanced Rules Governing Data-Flow Diagramming

Advanced Rules Governing Data-Flow Diagramming

Q. A composite data flow on one level can be split into component data flows at the next level, but no new data can be added, and all data in the composite must be accounted for in one or more subflows.

R. The input to a process must be sufficient to produce the outputs (including data placed in data stores) from the process. Thus, all outputs can be produced, and all data in inputs move somewhere, either to another process or to a data store outside the process or on a more detailed DFD showing a decomposition of that process.

S. At the lowest level of DFDs, new data flows may be added to represent data that are transmitted under exceptional conditions; these data flows typically represent error messages (e.g., “Customer not known; do you want to create a new customer?”) or confirmation notices (e.g., “Do you want to delete this record?”).

T. To avoid having data-flow lines cross each other, you may repeat data store or sources/sinks on a DFD. Use an additional symbol, like a double line on the middle vertical line of a data-store symbol, or a diagonal line in a corner of a source/sink square, to indicate a repeated symbol.

List of Abilities against which all projects are measured

"_____ability"

In the context of the Software Development Life Cycle (SDLC), projects are often evaluated against key "abilities" that determine their success. Following is a list of 16 essential abilities against which all projects can be measured:

1. ***Feasibility*** – The project must be realistic in terms of Economic, Operational, Technical, Schedule, Legal/Contractual, and Political viability.
2. ***Usability*** – The system should be user-friendly, intuitive, and easy to navigate.
3. ***Scalability*** – The software must be capable of handling growth in users, data, and transactions.
4. ***Reliability*** – The system should perform consistently under expected conditions without failure.
5. ***Maintainability*** – The software should be easy to update, modify, and fix without excessive effort.
6. ***Security*** – The system must protect data and operations from unauthorized access and threats.
7. ***Availability*** – The system should have minimal downtime and be accessible when needed.
8. ***Portability*** – The software should be able to run across different platforms and environments.
9. ***Interoperability*** – The system must integrate well with other applications and systems.
10. ***Testability*** – The software should allow for thorough testing to ensure quality and correctness.
11. ***Extensibility*** – The system should be designed to allow for future expansion and feature additions.
12. ***Adaptability*** – The software must be flexible enough to accommodate changes in requirements.
13. ***Performance*** – The system should meet speed, efficiency, and resource consumption expectations.
14. ***Auditability*** – The system should provide logs and tracking for compliance and troubleshooting.
15. ***Compliance*** – The software must meet industry regulations, legal requirements, and standards.
16. ***Affordability*** – The project should stay within budget while delivering value.

These abilities help analysts and designers evaluate the success and sustainability of projects across different SDLC phases, ensuring they meet stakeholder needs and long-term goals.

Enterprise Architecture and its 4 components

Enterprise Architecture is a holistic view of an organization's business, information, applications, and technology components, consisting of four key components: Business Architecture, Data Architecture, Application Architecture, and Technology Architecture; each representing a different aspect of the organization's operations and how they interact with each other.

Breakdown of the 4 components:

Business Architecture:
Defines the organization's strategic goals, processes, functions, and key stakeholders, outlining how the business operates at a high level and how different business units are aligned with overall strategy.

Data Architecture:
Describes the structure, organization, and relationships of data within the enterprise, including data models, data quality standards, and data governance policies.

Application Architecture:
Defines the design and interactions between applications within the enterprise, including how applications access and utilize data, and the overall application landscape.

Technology Architecture:
Details the underlying technology infrastructure that supports the applications and data, including hardware, software, networks, and operating systems

Key Terms and Definitions Chapter 6

1. ***Action stubs-*** That part of a decision table that lists the actions that result for a given set of conditions.
2. ***Balancing -*** The conservation of inputs and outputs to a data-flow diagram process when that process is decomposed to a lower level.
3. ***Condition stubs-*** That part of a decision table that lists the conditions relevant to the decision.
4. ***Context diagram-*** A data-flow diagram of the scope of an organizational system that shows the system boundaries, external entities that interact with the system, and the major information flows between the entities and the system**.**
5. ***Data store-*** Data at rest, which may take the form of many different physical representations.
6. ***Decision table -*** A matrix representation of the logic of a decision, which specifies the possible conditions for the decision and the resulting actions.
7. ***DFD completeness-*** The extent to which all necessary components of a data-flow diagram have been included and fully described.
8. ***DFD consistency-*** The extent to which information contained on one level of a set of nested data-flow diagrams is also included on other levels.
9. ***Gap analysis-*** The process of discovering discrepancies between two or more sets of data-flow diagrams or discrepancies within a single DFD**.**
10. ***Indifferent condition-*** In a decision table, a condition whose value does not affect which actions are taken for two or more rules.
11. ***Level-0 diagram-*** A data-flow diagram that represents a system's major processes, data flows, and data stores at a high level of detail.
12. ***Level-n diagram-*** A DFD that is the result of n nested decompositions of a series of subprocesses from a process on a level-0 diagram.
13. ***Primitive DFD-*** The lowest level of decomposition for a data-flow diagram.
14. ***Process-*** The work or actions performed on data so that they are transformed, stored, or distributed.
15. ***Process modeling-*** Graphically representing the processes that capture, manipulate, store, and distribute data between a system and its environment and among components within a system.
16. ***Rules-*** That part of a decision table that specifies which actions are to be followed for a given set of conditions**.**
17. ***Source/ sink-*** The origin and/or destination of data; sometimes referred to as external entities.

WEEK 8

7 Rules for Creating ERD's

1. ***Nouns and Verbs***- Entities are written as (singular) nouns (entity instance vs. entity); relationships are written as verbs
2. ***Cardinality***- Numerical relationship between rows of one table and rows in another
3. ***Primary Keys***- (1) unique, (2) not null, (3) include as few attributes as possible, (4) don't change over time
4. ***Mandatory Side***- Donates its key to the many side
5. ***Many to Many Relationship***- Associative entity is needed
6. ***Super Type/Sub Type***- Sub-types can exist when instances share some but not all attributes OR a set of instances have a unique relationship
7. ***Reference Table***- The same (non-key) data point should only exist once and in one entity

Key Terms and Definitions Chapter 7

1. ***Associative entity -*** An entity type that associates the instances of one or more entity types and contains attributes that are peculiar to the relationship between those entity instances
2. ***Attribute-*** A named property or characteristic of an entity that is of interest to the organization
3. ***Binary relationship-***A relationship between instances of two entity types
4. ***Candidate key-*** An attribute (or combination of attributes) that uniquely identifies each instance of an entity type
5. ***Cardinality-*** The number of instances of entity B that can (or must) be associated with each instance of entity A
6. ***Conceptual data model-*** A detailed model that shows the overall structure of organizational data while being independent of any database management system or other implementation considerations
7. ***Degree-*** The number of entity types that participate in a relationship.
8. ***Design strategy-*** A particular approach to developing an information system. It includes statements on the system's functionality, hardware and system software platform, and method for acquisition.
9. ***Entity-*** A person, place, object, event, or concept in the user environment about which the organization wishes to maintain data
10. ***Entity instance (instance) -*** A single occurrence of an entity type
11. ***Entity-relationship (E-R) diagram-*** A graphical representation of the entities, associations, and data for an organization or business area; it is a model of entities, the associations among those entities, and the attributes of both the entities and their associations
12. ***Entity type -*** A collection of entities that share common properties or characteristics
13. ***Identifier-*** A candidate key that has been selected as the unique, identifying characteristic for an entity type.
14. ***Multivalued attribute-*** An attribute that may take on more than one value for each entity instance.
15. ***Relationship -*** An association between the instances of one or more entity types that is of interest to the organization
16. ***Repeating group-*** A set of two or more multivalued attributes that are logically related
17. ***Relationship-*** An association between the instances of one or more entity types that is of interest to the organization.
18. ***Ternary relationship-*** A simultaneous relationship among instances of three entity types
19. ***Unary (recursive) relationship-*** A relationship between the instances of one entity type.

WEEK 11

The process for creating the UI

1. Requirements Gathering
Identify users: Who will use the system? (e.g., customers, employees, admins)
Define tasks: What tasks will users need to perform?
Understand user goals: What problems are you solving for the user?
Collect business rules and constraints that impact the UI.

2. **User and Task Analysis**
Develop personas to represent different types of users.
Create use case scenarios to map out user interactions.
Build task flows to outline step-by-step activities users will perform.

3. Define UI Requirements
List functional requirements related to the UI (e.g., search functionality, form submissions).
Include non-functional requirements (e.g., usability, accessibility, responsive design).
Consider platform and device constraints (laptop, mobile).

4. Design Prototypes
Sketch wireframes: Rough sketches of screens to visualize layout.
Low-fidelity prototypes: Basic, minimal digital representation of a user interface or system. High-fidelity prototypes (Mock-Up): Preview of a page or screen before the system is fully built

5. Apply UI Design Principles
Consistency: Maintain visual and functional consistency across the system.
Affordance: Ensure UI elements clearly indicate how to interact with them.
Feedback: Provide users with feedback for actions (e.g., loading spinners, success messages).
Error Prevention and Recovery: Design to reduce user errors and allow easy recovery.

6. User Validation
Conduct user testing with prototypes.
Gather feedback and observe how users interact with the design.
Iterate based on user input.

7. Refine and Finalize UI
Adjust designs based on feedback and align with system and business goals.
Ensure accessibility (laptop, mobile).
Finalize a UI specification document to hand off to developers.

8. Handoff to Development

9. Post-Implementation Testing

The steps for creating the UX

1. Research
User research: Interviews, surveys, observations, user personas.
Competitive analysis: Look at how others solve similar problems.
Stakeholder interviews: Understand business goals and constraints.
Data analysis: Use analytics to identify patterns and behaviors.

2. Define
Problem statement: What's the user problem you're solving?
User personas: Create fictional representations of your key users.
User journey mapping: Map out the user's flow and identify pain points.
Requirements: Both functional and non-functional

3. Ideation
Brainstorming: Generate ideas for solutions.
Sketching: Quick and rough visualizations of ideas.
User flow diagrams: Define how users will navigate through the product.
Information architecture: Organize content and structure.

4. Prototyping
Wireframes: Basic, low-fidelity layouts of screens/pages.
Clickable prototypes: Interactive models
Mid to high-fidelity designs: Adding more detail

5. Testing
Usability testing: Get real users to test the prototype.
A/B testing: Compare two versions to see which works better.
Collect feedback: Analyze what's working and what's not.

6. Implementation
Work closely with developers to ensure the UX is properly implemented.
Design handoff: Provide assets, specs, and guidance.

7. Launch & Monitor

8. Iterate - UX is never "done."

The 3 things wireframing allows the designer to plan

Wireframing allows the designer to plan three key things:

Layout and Structure – It helps map out the basic arrangement of elements on a page or screen, including navigation, content areas, buttons, and images.

User Flow and Interaction – Designers can visualize how users will navigate through the interface and how different elements will interact (e.g., links, buttons, forms).

Content Placement and Hierarchy – Wireframing allows designers to decide where and how much content to place, ensuring important information is prioritized and the user experience is clear and intuitive.

Key characteristics of low-fidelity prototyping

- Minimal detail: Basic shapes, placeholders, etc.
- Monochrome or grayscale: Typically, black-and-white or shades of gray.
- Focus on flow, not aesthetics: Emphasizes screen layout, and interaction points, such as buttons, links, etc.
- Interactive: Often clickable or tappable to simulate navigation, such as moving from one screen to another.
- Fast and inexpensive: Quick to create, easy to adjust based on feedback.

The 3 components of a high-fidelity prototype (mock-up)

1. **Layout & Structure** - Shows where different UI elements like buttons, menus, forms, images, and text will be placed.
2. **Visual Design** - Can include colors, typography, branding, & other design elements.
3. **User Flow** - Helps illustrate how users will interact with the system by showing multiple screens and how they connect

Key elements of screen flow

- **Screens/pages**: Each box or node represents a distinct screen, form, or interface.
- **Transitions**: Arrows or connectors show how users move from one screen to another
- **Decision points**: Conditional logic, showing alternative paths depending on user choices or system rules.

Key terms and definitions from Chapter 8

1. **Audit trail-** *A record of the sequence of data entries and the date of those entries*

2. **Cookie crumbs-** *A technique for showing users where they are in a website by placing a series of "tabs" on a Web page that shows users where they are and where they have been.*

3. **Dialogue-** *The sequence of interaction between a user and a system.*

4. **Dialogue diagramming-** *A formal method for designing and representing human-computer dialogues using box-and-line diagrams*

5. **Form-** *A business document that contains some predefined data and may include some areas where additional data are to be filled in; typically based on one database record.*

6. **Lightweight graphics-** *A technique for showing users where they are in a website by placing a series of "tabs" on a Web page that shows users where they are and where they have been.*

7. **Report-** *A business document that contains only predefined data; it is a passive document used only for reading or viewing; typically contains data from many unrelated records or transactions.*

8. **Style sheet–based HTML-** *Templates to display and process common attributes of higher-level, more abstract items.*

WEEK 12

0NF, 1NF, 2NF, 3NF identification guidelines

Normal Forms

0NF

- Something that exists in the form of a table.
- Not Atomic *(A comma or semicolon is a dead giveaway!!)*

GuideID*	Name	DOB	HourlyRt	Skills	KnownLakes
1	Kim Deal	4/12/1970	50	Knot Tying, Dog Training, Good with Kids	Emerald, Cascade, Glacier, Keyser Brown
2	Charles Thompson	8/1/1968	45	Fire Building, Rock Climbing, Screaming	Rosebud, Fox
3	Joey Santiago	9/14/1969	47.50	Bear Avoidance, Pathfinding, Story Telling	Glacier, First Rock
4	David Lovering	1/30/1967	40	Good with Kids, Story Telling	First Rock, Rosebud

** = Primary Key, † = Foreign Key*

Not Atomic!

TGuide(GuideID, GName, GDOB, GHourlyRt, GSkills, GKnownLakes)

1NF

- Atomic (no multi-value or composite attributes)
- Has partial functional dependencies. *This NF only happens when we have a composite PK and some of the attributes depend on part of the PK, not the whole PK.*

CustID*	FName	LName	Email	SOID*	Date	EmpID†
11	Edgar	Martinez	edgardh@hotmail.com	101	5/11/15	6
18	Jay	Buhner	buzz@yahoo.com	102	5/11/15	4
25	Chris	Bosio	cbos@msn.com	103	5/11/15	7
6	Dan	Wilson	danthemanm@hotmail.com	104	5/12/15	6
29	Vince	Coleman	vccf@yahoo.com	105	5/12/15	4
11	Edgar	Martinez	edgardh@hotmail.com	106	5/12/15	3

** = Primary Key, † = Foreign Key*

TCust(CustID, CFName, CLName, CEmail, CSOID, CDate, CEmpID)

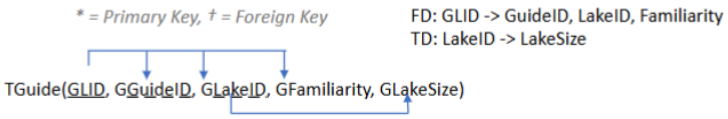
PFD: CustID -> FName, LName, Email
TD: SOID -> CustID, EmpID, Date

2NF = 1NF +

- No partial functional dependencies
- Has transitive dependencies. *Non-key attribute predicts other non-key attributes*.

GLID*	GuideID†	LakeID†	Familiarity	LakeSize
1	1	3	5	150
2	2	3	2	150
3	1	7	4	96
4	1	10	3	400
5	2	7	1	96
6	3	12	4	40
7	3	7	3	96

Predicted by less than the full PK!



3NF = 2NF +

- No transitive dependencies.

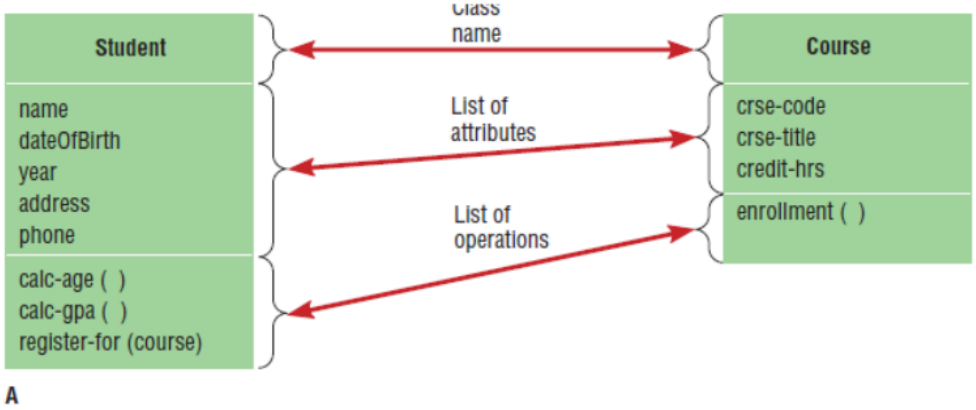
Key terms and definitions from Chapter 9

1. ***Relation-*** A field that can be derived from other database fields.
2. ***Well-structured relation-*** A relation that contains a minimum amount of redundancy and allows users to insert, modify, and delete the rows without errors or inconsistencies.
3. ***Normalization-*** The process of converting complex data structures into simple, stable data structures.
4. ***Functional dependency-*** A particular relationship between two attributes. For a given relation, attribute B is functionally dependent on attribute A if, for every valid value of A, that value of A uniquely determines the value of B. The functional dependence of B on A is represented by A→B
5. ***Second normal form-*** A relation for which every nonprimary key attribute is functionally dependent on the whole primary key.
6. ***Third normal form-*** A relation that is in second normal form and has no functional (transitive) dependencies between two (or more) nonprimary key attributes.
7. ***Foreign key-*** An attribute that appears as a nonprimary key attribute in one relation and as a primary key attribute (or part of a primary key) in another relation.
8. ***Referential integrity-*** An integrity constraint specifying that the value (or existence) of an attribute in one relation depends on the value (or existence) of the same attribute in another relation.
9. ***Recursive foreign key-*** A foreign key in a relation that references the primary key values of that same relation.
10. ***Synonyms-*** Two different names that are used for the same attribute
11. ***Homonym-*** A single attribute name that is used for two or more different attributes
12. ***Field-*** The smallest unit of named application data recognized by system software
13. ***Data type-*** A coding scheme recognized by system software for representing organizational data
14. ***Calculated field-*** A field that can be derived from other database fields.
15. ***Default value-*** The value a field will assume unless an explicit value is entered for that field.
16. ***Input mask-*** A pattern of codes that restricts the width and possible values for each position of a field
17. ***Null value-*** A special field value, distinct from 0, blank, or any other value, that indicates that the value for the field is missing or otherwise unknown
18. ***Physical table-*** A named set of rows and columns that specifies the fields in each row of the table
19. ***Denormalization-*** The process of splitting or combining normalized relations into physical tables based on affinity of use of rows and fields.
20. ***Physical file-*** A named set of table rows stored in a contiguous section of secondary memory
21. ***File organization-*** A technique for physically arranging the records of a file
22. ***Pointer-*** A field of data that can be used to locate a related field or row of data
23. ***Sequential file organization-*** The rows in the file are stored in sequence according to a primary key value
24. ***Indexed file organization-*** The rows are stored either sequentially or non sequentially, and an index is created that allows software to locate individual rows
25. ***Index-*** A table used to determine the location of rows in a file that satisfy some condition
26. ***Secondary key-*** One or a combination of fields for which more than one row may have the same combination of values.

27. ***Hashed file organization-*** The address for each row is determined using an algorithm
28. ***Primary key-*** An attribute whose value is unique across all occurrences of a relation
29. ***Relational database model-*** Data represented as a set of related tables or relations.

WEEK 13

CLASS DIAGRAM

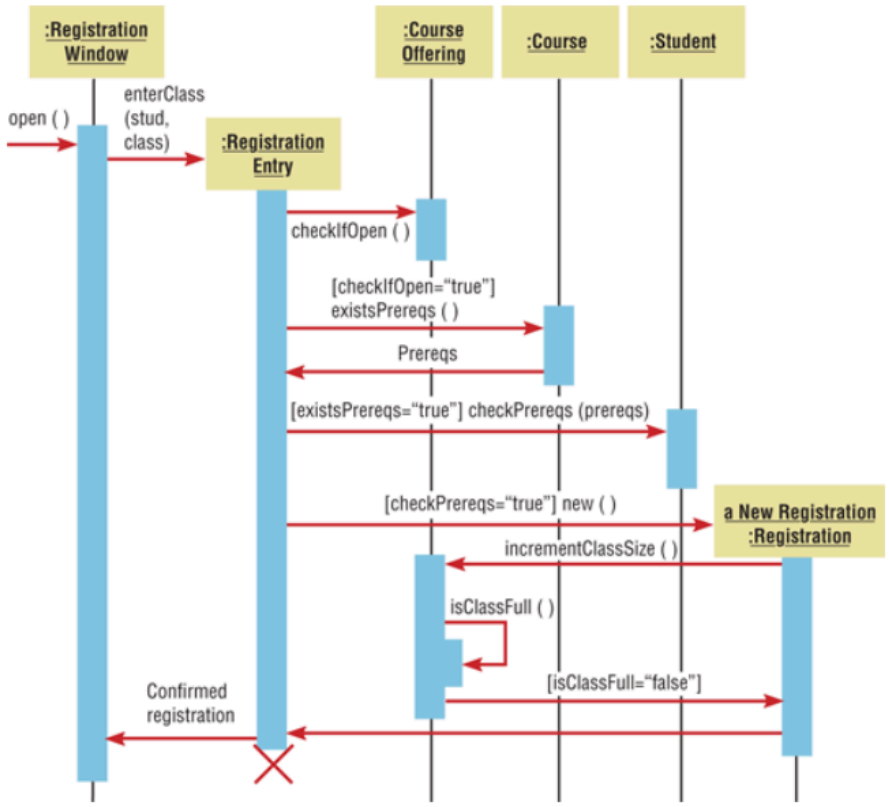


A

Use case narrative:

The class diagram represents a system for managing student registrations and course enrollments. In this system, the **Student** class captures details such as the student's name, date of birth, year, address, and phone number. The **Student** class also includes operations to calculate the student's age (`calc-age()`) and GPA (`calc-gpa()`), which can be useful for academic tracking and eligibility determinations. On the other hand, the **Course** class contains attributes like the course code, title, and credit hours, and includes an operation for enrollment (`enrollment()`) to track which students are registered in each course. The core use case involves a **Student** registering for a **Course** by invoking the `register-for(course)` operation, which links the student to the course and updates the enrollment records. This system facilitates managing student information, course details, and the registration process, ensuring that students are properly enrolled in the courses they wish to take.

SEQUENCE DIAGRAM



Use case:

In response to the "open" message from the Registration Clerk (external actor), the Registration Window pops up on the screen, and the registration information is entered. A new Registration Entry object is created, which then sends a checkIfOpen message to the Course Offering object (representing the class the student wants to register for). The two possible return values are true or false. In this scenario, the assumption is that the class is open. We have therefore placed a guard condition, checkIfOpen = "true," on the message existsPrereqs. The guard condition ensures that the message will be sent only if the class is open. The return value is a list of prerequisites; the return is shown explicitly in the diagram.

For this scenario, the fact that the course has prerequisites is captured by the guard condition, existsPrereqs = "true." If this condition is satisfied, the Registration Entry object sends a checkPrereqs message, with "prereqs" as an argument, to the Student object to determine if the student has taken those prerequisites. If the student has taken all the prerequisites, the Registration Entry object creates an object called a New Registration, which denotes a new registration.

Next, a New Registration sends a message called incrementClassSize to Course Offering in order to increase the class size by one. The incrementClassSize operation within Course Offering then calls upon isClassFull, another operation within the same object. Assuming that the class is not full, the isClassFull

operation returns control to the calling operation with a value of “false.” Next, the incrementClassSize operation completes and relinquishes control to the calling operation within a New Registration.

Finally, on receipt of the return message from a New Registration, the Registration Entry object destroys itself (the destruction is shown with a large X) and sends a confirmation of the registration to the Registration Window. Note that Registration Entry is not a persistent object; it is created on the fly to control the sequence of interactions and is deleted as soon as the registration is completed. In between, it calls several other operations within other objects by sequencing the following messages: checkIfOpen, existsPrereqs, checkPrereqs, and new.

Apart from the Registration Entry object, a New Registration is also created during the time period captured in the diagram. The messages that created these objects are represented by arrows pointing directly toward the object symbols. For example, the arrow representing the message called new is connected to the object symbol for a New Registration. The lifeline of such an object begins when the message that creates it is received (the vertical line is hidden behind the activation rectangle).

Key Vocabulary Appendix A

1. ***Abstract class-***A class that has no direct instances but whose descendants may have direct instances.
2. ***Activation-***The time period during which an object performs an operation
3. ***Actor-***An external entity that interacts with the system (similar to an external entity in data-flow diagramming).
4. ***Aggregation-***A part-of relationship between a component object and an aggregate object
5. ***Association-***A relationship among object classes
6. ***Association role-***The end of an association where it connects to a class
7. ***Behavior-*** Represents how an object acts and reacts
8. ***Class diagram-***A diagram that shows the static structure of an object-oriented model: the object classes, their internal structure, and the relationships in which they participate
9. ***Component diagram-*** A diagram that shows the software components or modules and their dependencies
10. ***Concrete class-***A class that can have direct instances
11. ***Encapsulation-*** The technique of hiding the internal implementation details of an object from its external view
12. ***Event-***Something that takes place at a certain point in time; it is a noteworthy occurrence that triggers a state transition
13. ***Multiplicity-***An indication of how many objects participate in a given relationship
14. ***Object-***An entity that has a well-defined role in the application domain and has state, behavior, and identity.
15. ***Object class-***A set of objects that shares a common structure and a common behavior
16. ***Object diagram-***A graph of instances that are compatible with a given class diagram
17. ***Operation-*** A function or a service that is provided by all the instances of a class
18. ***Sequence diagram-*** A depiction of the interactions among objects during a certain period of time
19. ***Simple message -*** A message that transfers control from the sender to the recipient without describing the details of the communication
20. ***State-***A condition that encompasses an object's properties (attributes and relationships) and the values those properties have.
21. ***State transition-***The changes in the attributes of an object or in the links an object has with other objects
22. ***Synchronous message-***A type of message in which the caller has to wait for the receiving object to finish executing the called operation before it can resume execution itself
23. ***Unified Modeling Language (UML)-*** A notation that allows the modeler to specify, visualize, and construct the artifacts of software systems, as well as business models.
24. ***Use case-***A sequence of related actions initiated by an actor; it represents a specific way to use the system
25. ***Use-case diagram-*** A diagram that depicts the use cases and actors for a system.

WEEK 14

Sample Test Plan

Test Plan for Internship Management System

1. Introduction

This test plan outlines the approach for testing the Internship Management System developed as part of a Systems Analysis and Design (SAD) course project. The system allows students to view and apply for internships, and employers to post and manage internship listings. The purpose is to ensure that the application is functional, secure, user-friendly, and meets the course requirements.

---

2. Objectives

- Validate functionality for all major user flows: posting, searching, applying, and reviewing internships.
- Ensure data integrity between student, employer, and internship records.
- Confirm integration with school authentication (SSO) and email systems.
- Verify role-based access and UI usability for students, employers, and administrators.

---

3. Scope

In-Scope Testing:

- Internship listing and search features
- Application submission and status tracking
- Employer and student profile management
- Admin capabilities for approval and reporting
- Email notifications (application submitted, status change)

Out-of-Scope Testing:

- External HR systems integration
- Post-graduation employment tracking

---

4. Test Strategy

Test Levels:

- **Unit Testing**: Validate input fields, profile updates, login/authentication.
- **Integration Testing**: Ensure modules (e.g., application form + email service) work together.
- **System Testing**: Simulate a full process: student applies → employer reviews → admin tracks.
- **User Acceptance Testing (UAT)**: Performed with peers and instructors.

Types of Testing:

- **Functional Testing**: Test login, application workflows, search filters.
    - *Expected Outcome:* System accepts valid inputs, returns correct results.
- **Performance/Load Testing**:
    - *Performance:* Pages load under 2 seconds on average.
    - *Load:* System handles 50 simultaneous users without error.
- **Security Testing**: Ensure that student data is not accessible to unauthorized users.
    - *Expected Outcome:* Only authorized roles can access respective dashboards.
- **Usability Testing**: Evaluate intuitiveness of student and employer dashboards.
    - *Expected Outcome:* Users complete tasks (e.g., applying to an internship) in under 3 minutes.
- **Regression Testing**: After changes like UI updates or new features (e.g., resume uploads).

- *Expected Outcome:* Previous features still function correctly.

---

5. Test Environment

- Web-based system hosted in a staging environment (e.g., Heroku or school server).
- Mock student and employer accounts.
- Pre-populated database with test internships and profiles.

---

6. Test Scenarios and Cases

Scenario	Test Case	Expected Outcome
Student registration and login	Input valid student ID and password	Login successful, redirect to student dashboard
Internship search	Search by location, major, or company	Matching results are displayed correctly
Apply for internship	Fill and submit application form	Confirmation email sent, status shown as "Submitted"
Employer posts internship	Complete posting form	Internship appears in student search results
Admin reviews applications	View student applications per internship	Status updates trigger email to student

---

7. Roles and Responsibilities

Role	Responsibility
Student QA Lead	Write and execute test cases, report bugs
Project Developer	Address issues, provide build updates
Systems Analyst	Validate requirements and testing scope

Instructor	Oversee progress and review test documentation
Peer Reviewer	Simulate student/employer use during UAT

---

8. Defect Management

- Defects tracked in **GitHub Issues** or **Trello**.
- Severity Levels: Critical, High, Medium, Low.
- Daily check-ins during development sprint to prioritize and resolve.

---

9. Entry and Exit Criteria

Entry Criteria:

- All features implemented.
- Unit tests passed.
- Testing environment set up.

Exit Criteria:

- No critical/high bugs open.
- UAT complete with sign-off.
- Final test summary submitted to instructor.

---

10. Deliverables

- Complete Test Plan Document (this file)
- Test Cases and Results (Excel sheet or Trello board)

- Bug Reports
- UAT Feedback Summary
- Final Testing Report

Key Terms and Definitions Chapter 10

1. ***Inspections***-A testing technique in which participants examine program code for predictable language-specific errors.
2. ***Desk checking***-A testing technique in which the program code is sequentially executed manually by the reviewer.
3. ***Electronic performance support system-***Component of a software package or application in which training and educational information is embedded. An EPSS may include a tutorial, expert system, and hypertext jumps to reference material.
4. ***User documentation-***Written or other visual information about how an application system works, and how to use it
5. ***Direct installation-***Changing over from the old information system to a new one by turning off the old system when the new one is turned on
6. ***Adaptive maintenance-***Changes made to a system to evolve its functionality to changing business needs or technologies
7. ***Unit testing-***Each module is tested alone in an attempt to discover any errors in its code.
8. ***Installation-***The organizational process of changing over from the current information system to a new one
9. ***Mean time between failures-***A measurement of error occurrences that can be tracked over time to indicate the quality of a system
10. ***External documentation-***System documentation that includes the outcome of structured diagramming techniques such as data-flow and entity-relationship diagrams
11. ***Acceptance testing-***The process whereby actual users test a completed information system, the end result of which is the users' acceptance of it once they are satisfied with it
12. ***Build routines-***Guidelines that list the instructions to construct an executable system from the baseline source code
13. ***Preventive maintenance-***Changes made to a system to avoid possible future problems
14. ***System documentation-***Detailed information about a system's design specifications, its internal workings, and its functionality
15. ***Parallel installation-***Running the old information system and the new one at the same time until management decides the old system can be turned off
16. ***Integration testing-***The process of bringing together for testing purposes all of the modules that a program comprises. Modules are typically integrated in a top-down, incremental fashion.
17. ***Perfective maintenance-***Changes made to a system to add new features or to improve performance.
18. ***Stub testing-***A technique used in testing modules, especially where modules are written and tested in a top-down fashion, where a few lines of code are used to substitute for subordinate modules.
19. ***Baseline modules-***Software modules that have been tested, documented, and approved to be included in the most recently created version of a system
20. ***System librarian-***A person responsible for controlling the checking out and checking in of baseline modules when a system is being developed or maintained

21. ***Phased installation-***Changing from the old information system to the new one incrementally, starting with one or a few functional components and then gradually extending the installation to cover the whole new system
22. ***Configuration management-***The process of ensuring that only authorized changes are made to a system
23. ***System testing-***The bringing together for testing purposes of all the programs that a system comprises. Programs are typically integrated in a top-down, incremental fashion
24. ***Maintenance-***Changes made to a system to fix or enhance its functionality
25. ***Internal documentation-***System documentation that is part of the program source code or is generated at compile time
26. ***Support-***Providing ongoing educational and problem-solving assistance to information system users. Support material and jobs must be designed along with the associated information system
27. ***Beta testing-***User testing of a completed information system using real data in the real user environment
28. ***Corrective maintenance-***Changes made to a system to repair flaws in its design, coding, or implementation
29. ***Single location installation-***Trying out a new information system at one site and using the experience to decide if and how the new system should be deployed throughout the organization
30. ***Alpha testing-***User testing of a completed information system using simulated data
31. ***Help desk-***A single point of contact for all user inquiries and problems about a particular information system or for all users in a particular department
32. ***Testing harness-***An automated testing environment used to review code for errors, standards violations, and other design flaws.
33. ***Issue tracking software-***Typically a Web-based tool for logging, tracking, and assigning system bugs and change requests to developers.

WEEK 15

Purpose of a Disaster Recovery Plan and Sample Disaster Recovery Plan

DISASTER RECOVERY PLAN

Purpose
Disaster Recovery focuses specifically on restoring IT systems, applications, and data following a disruptive event. It is triggered by cyberattacks, hardware failures, etc. The goal is to minimize data loss and downtime.
Define the Scope
IT infrastructure: AWS-hosted servers, internship project database, student-facing web portal

Application systems: Web-based student advising and internship tracking system

Data backups and restoration: Daily full + hourly incremental backups to S3

RTO (Recovery Time Objective): 6 hours

RPO (Recovery Point Objective): 1 hour

Steps to Build a Disaster Recovery Plan
1. Conduct Risk Assessment & Business Impact Analysis (BIA)
Identify threats:

Cyberattacks

Accidental deletions

Cloud platform outages

Determine critical systems:

Internship placement database

Login/authentication service

Email and notification systems

Analyze impact of downtime:

Missed placement deadlines

Disrupted student-advisor communication

2. Define Recovery Objectives
RTO: 6 hours

RPO: 1 hour

3. Inventory Resources & Dependencies
Hardware: AWS EC2, RDS, S3

Software: Internship web app, admin dashboard

Network: University VPN, campus Wi-Fi

Third-party providers:

Firebase (for auth)

SendGrid (email)

4. Develop Recovery Strategies
Backup methods:

Daily full backups (overnight)

Hourly incremental backups

Redundant systems:

Multi-zone AWS setup

Warm standby deployment in another region

Cloud-based recovery:

AWS AMI snapshots

S3 versioning

5. Create Recovery Procedures
Step-by-step restoration:

Notify DR team via Slack & email

Redirect DNS to standby servers

Restore most recent backup from S3

Test app functionality

Contact lists:

Project Lead, IT Admin, Faculty Sponsor

Access protocols:

MFA on all critical cloud accounts

6. Test the DR Plan
Monthly simulated failover drill

Quarterly test restore from backup

7. Maintain and Update
Review after every deployment or architecture change

Conduct semesterly staff training and updates

Business Continuity Plan

BUSINESS CONTINUITY PLAN
Purpose
Business Continuity ensures that entire business operations continue during and after a disruption, not just IT systems. It covers people, facilities, logistics, etc. The focus is on ensuring all operations continue.
Scope
People: Student developers, advisors, course instructor

Processes: Intern tracking, evaluations, student-advisor communications

Facilities: Campus computer labs, project server access

Communications and logistics: Slack, Zoom, email updates

Steps to Build a Business Continuity Plan
1. Establish a Business Continuity Team
Assign roles:

Project Manager (student lead)

IT Coordinator (developer)

Communication Officer (liaison to faculty)

Define responsibilities and authority for each team member

2. Perform Business Impact Analysis (BIA)
Identify essential functions:

Student record updates

Submission of evaluations

Placement communications

Rank priority:

Real-time updates: high priority

Report generation: moderate

Tolerance:

24 hours maximum delay on core functions

3. Identify and Assess Risks
Risk scenarios:

Campus closure (e.g., weather)

Loss of network access

Key personnel unavailable

Evaluate probability and consequences

4. Develop Business Continuity Strategies
Remote operations:

Zoom and Teams for all team meetings

Web app access via VPN

Manual alternatives:

Use of shared Google Sheets for tracking

Email chains for critical updates

Partner and stakeholder planning:

Faculty advisors briefed on alternate process

Clear communication protocols for students

5. Create Continuity Procedures
Evacuation:

Follow campus alert system guidelines

Communication:

Use Slack, email, and emergency phone tree

Departmental checklist:

Confirm app/server status

Send outage notice to users

Trigger manual tracking if needed

Contingency staffing:

Assistant project lead steps in for manager

Shared documentation ensures smooth handoff

6. Training and Awareness
Conduct tabletop exercise each semester

Provide documentation to all team members

Run continuity drill during project midterm phase

7. Plan Maintenance and Review
Update plan post-deployment and after team turnover

Set reminders to review plan quarterly

Definitions

1. Business Impact Analysis (BIA)- A process that identifies and evaluates how disruptions (like disasters or outages) would affect a business's operations, helping prioritize which systems and processes need to be restored first.
2. Recovery Time Objective (RTO)- The maximum amount of time a business can afford for a system or process to be down after a disruption before it seriously impacts operations.
3. Recovery Point Objective (RPO)-The maximum acceptable amount of data loss measured in time; it defines how far back in time the business can go to recover data after an outage (for example, losing the last 4 hours of data)